

```
pip install pandas scikit-learn matplotlib seaborn wordcloud nltk  
openpyxl
```

Collecting pandas

Downloading pandas-3.0.0-cp311-cp311-macosx_11_0_arm64.whl.metadata
(79 kB)

Collecting scikit-learn

Downloading scikit_learn-1.8.0-cp311-cp311-
macosx_12_0_arm64.whl.metadata (11 kB)

Collecting matplotlib

Downloading matplotlib-3.10.8-cp311-cp311-
macosx_11_0_arm64.whl.metadata (52 kB)

Collecting seaborn

Using cached seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)

Collecting wordcloud

Downloading wordcloud-1.9.6-cp311-cp311-macosx_11_0_arm64.whl.metadata
(3.4 kB)

Collecting nltk

Using cached nltk-3.9.2-py3-none-any.whl.metadata (3.2 kB)

Collecting openpyxl

Using cached openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)

Collecting numpy>=1.26.0 (from pandas)

Downloading numpy-2.4.2-cp311-cp311-macosx_14_0_arm64.whl.metadata
(6.6 kB)

Requirement already satisfied: python-dateutil>=2.8.2 in

./venv/lib/python3.11/site-packages (from pandas) (2.9.0.post0)

Collecting scipy>=1.10.0 (from scikit-learn)

Downloading scipy-1.17.0-cp311-cp311-macosx_14_0_arm64.whl.metadata
(62 kB)

Collecting joblib>=1.3.0 (from scikit-learn)

Using cached joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)

Collecting threadpoolctl>=3.2.0 (from scikit-learn)

Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)

Collecting contourpy>=1.0.1 (from matplotlib)

Downloading contourpy-1.3.3-cp311-cp311-macosx_11_0_arm64.whl.metadata
(5.5 kB)

Collecting cyclor>=0.10 (from matplotlib)

Using cached cyclor-0.12.1-py3-none-any.whl.metadata (3.8 kB)

Collecting fonttools>=4.22.0 (from matplotlib)

Downloading fonttools-4.61.1-cp311-cp311-
macosx_10_9_universal2.whl.metadata (114 kB)

Collecting kiwisolver>=1.3.1 (from matplotlib)

Downloading kiwisolver-1.4.9-cp311-cp311-
macosx_11_0_arm64.whl.metadata (6.3 kB)

Requirement already satisfied: packaging>=20.0 in

./venv/lib/python3.11/site-packages (from matplotlib) (26.0)

```
Collecting pillow>=8 (from matplotlib)
  Downloading pillow-12.1.1-cp311-cp311-macosx_11_0_arm64.whl.metadata
(8.8 kB)
Collecting pyparsing>=3 (from matplotlib)
  Using cached pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Collecting click (from nltk)
  Using cached click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting regex>=2021.8.3 (from nltk)
  Downloading regex-2026.1.15-cp311-cp311-macosx_11_0_arm64.whl.metadata
(40 kB)
Collecting tqdm (from nltk)
  Using cached tqdm-4.67.3-py3-none-any.whl.metadata (57 kB)
Collecting et-xmlfile (from openpyxl)
  Using cached et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.11/site-
packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Downloading pandas-3.0.0-cp311-cp311-macosx_11_0_arm64.whl (9.9 MB)
[2K   [38;2;114;156;31m-----[0m
[32m9.9/9.9 MB [0m [31m338.7 kB/s [0m [33m0:00:28 [0ma
[36m0:00:01 [0m[36m0:00:02 [0m:03 [0m
[?25hDownloading scikit_learn-1.8.0-cp311-cp311-macosx_12_0_arm64.whl
(8.1 MB)
[2K   [38;2;114;156;31m-----[0m
[32m8.1/8.1 MB [0m [31m252.0 kB/s [0m [33m0:00:32 [0m [31m252.8
kB/s [0m eta [36m0:00:01 [0m
[?25hDownloading matplotlib-3.10.8-cp311-cp311-macosx_11_0_arm64.whl
(8.1 MB)
[2K   [38;2;114;156;31m-----[0m
[32m8.1/8.1 MB [0m [31m306.2 kB/s [0m [33m0:00:26 [0m [0m eta
[36m0:00:01 [0m[36m0:00:01 [0m
[?25hUsing cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Downloading wordcloud-1.9.6-cp311-cp311-macosx_11_0_arm64.whl (168 kB)
Using cached nltk-3.9.2-py3-none-any.whl (1.5 MB)
Using cached openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Downloading contourpy-1.3.3-cp311-cp311-macosx_11_0_arm64.whl (270 kB)
Using cached cyclor-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.61.1-cp311-cp311-macosx_10_9_universal2.whl (2.9
MB)
[2K   [38;2;114;156;31m-----[0m
[32m2.9/2.9 MB [0m [31m240.1 kB/s [0m [33m0:00:11 [0m [31m235.6
kB/s [0m eta [36m0:00:01 [0m
[?25hUsing cached joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading kiwisolver-1.4.9-cp311-cp311-macosx_11_0_arm64.whl (65 kB)
Downloading numpy-2.4.2-cp311-cp311-macosx_14_0_arm64.whl (5.5 MB)
[2K   [38;2;114;156;31m-----[0m
[32m5.5/5.5 MB [0m [31m253.3 kB/s [0m [33m0:00:21 [0m6.7 kB/s [0m eta
[36m0:00:01 [0m:05 [0m
[?25hDownloading pillow-12.1.1-cp311-cp311-macosx_11_0_arm64.whl (4.7
MB)
[2K   [38;2;114;156;31m-----[0m
```

```

[32m4.7/4.7 MB [0m [31m213.8 kB/s [0m [33m0:00:21 [0m2.1 kB/s [0m eta
[36m0:00:01 [0m:04 [0m
[?25hUsing cached pyparsing-3.3.2-py3-none-any.whl (122 kB)
Downloading regex-2026.1.15-cp311-cp311-macosx_11_0_arm64.whl (288 kB)
Downloading scipy-1.17.0-cp311-cp311-macosx_14_0_arm64.whl (20.1 MB)
[2K [38;2;114;156;31m----- [0m
[32m20.1/20.1 MB [0m [31m327.8 kB/s [0m [33m0:01:10 [0m [0m eta
[36m0:00:01 [0m[36m0:00:03 [0m
[?25hUsing cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Using cached click-8.3.1-py3-none-any.whl (108 kB)
Using cached et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Using cached tqdm-4.67.3-py3-none-any.whl (78 kB)
Installing collected packages: tqdm, threadpoolctl, regex, pyparsing,
pillow, numpy, kiwisolver, joblib, fonttools, et-xmlfile, cycler, click,
scipy, pandas, openpyxl, nltk, contourpy, scikit-learn, matplotlib,
wordcloud, seaborn
[2K [38;2;114;156;31m----- [0m
[32m21/21 [0m [seaborn]— [0m [32m18/21 [0m [matplotlib]n]x1]
[1A [2KSuccessfully installed click-8.3.1 contourpy-1.3.3 cycler-0.12.1
et-xmlfile-2.0.0 fonttools-4.61.1 joblib-1.5.3 kiwisolver-1.4.9
matplotlib-3.10.8 nltk-3.9.2 numpy-2.4.2 openpyxl-3.1.5 pandas-3.0.0
pillow-12.1.1 pyparsing-3.3.2 regex-2026.1.15 scikit-learn-1.8.0 scipy-
1.17.0 seaborn-0.13.2 threadpoolctl-3.6.0 tqdm-4.67.3 wordcloud-1.9.6

[1m[ [0m [34;49mnotice [0m [1;39;49m] [0m [39;49m A new release of pip is
available: [0m [31;49m26.0 [0m [39;49m -> [0m [32;49m26.0.1 [0m
[1m[ [0m [34;49mnotice [0m [1;39;49m] [0m [39;49m To update, run:
[0m [32;49mpip install --upgrade pip [0m
Note: you may need to restart the kernel to use updated packages.

```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
import nltk
import re
import warnings
warnings.filterwarnings('ignore')

nltk.download('stopwords')
nltk.download('punkt')
from nltk.corpus import stopwords

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

```

```

from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score, f1_score)

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import LabelEncoder

print("All imports successful!")

```

All imports successful!

```

[nltk_data] Downloading package stopwords to
[nltk_data]      /Users/sharanm/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /Users/sharanm/nltk_data...
[nltk_data]   Package punkt is already up-to-date!

```

```

df = pd.read_csv("academic_documents.csv")
print(f"Dataset shape: {df.shape}")
print(f"\nColumns: {list(df.columns)}")
print(f"\nFirst 5 rows:")
df.head()

```

Dataset shape: (452, 6)

Columns: ['doc_id', 'filename', 'course_label', 'module_label', 'doc_type', 'file_format']

First 5 rows:

	doc_id	filename	course_label	module_label	doc_type	file_format
0	1	Data_Structures_and_Algorithm_Analysis_in_C.pdf	Data Structures and Algorithms	unknown	notes	pdf
1	2	CLRS-3rd Edition.pdf	Data Structures and Algorithms	unknown	notes	pdf
2	3	EBook-T1-Weiss-Datastructure(DSA).pdf	Data Structures and Algorithms	unknown	notes	pdf
3	4	Data Structures and Algorithms - Narasimha Kar...	Data Structures and Algorithms	unknown	notes	pdf

	doc_id	filename	course_label	module_label	doc_type	file_format
4	5	EBook-R2-Advanced Algorithms-ThomasH-Cormen(DS...	Data Structures and Algorithms	unknown	notes	pdf

```

print("=" * 50)
print("DATASET OVERVIEW")
print("=" * 50)
print(f"Total documents: {len(df)}")
print(f"\nCourse distribution:")
print(df['course_label'].value_counts())
print(f"\nDocument type distribution:")
print(df['doc_type'].value_counts())
print(f"\nFile format distribution:")
print(df['file_format'].value_counts())
print(f"\nMissing values:")
print(df.isnull().sum())

```

```

=====
DATASET OVERVIEW
=====
Total documents: 452

Course distribution:
course_label
Data Structures and Algorithms          98
Microprocessors and Microcontrollers    98
Computer Architecture and Organisation  94
Complex Variables and Linear Algebra    87
Operating Systems                      75
Name: count, dtype: int64

Document type distribution:
doc_type
notes          328
slides         122
syllabus         1
assignment         1
Name: count, dtype: int64

File format distribution:
file_format
pdf          326
pptx         122
docx           4
Name: count, dtype: int64

```

Missing values:

```
doc_id      0
filename     0
course_label 0
module_label 0
doc_type     0
file_format  0
dtype: int64
```

```
print("=" * 50)
print("DATA PREPROCESSING")
print("=" * 50)

# Drop rows with unknown/general module labels for classification
df_clean = df[~df['module_label'].isin(['General', 'Unknown',
                                         'FOR MODULE-2',
                                         'FOR MODULE-3 AND 4',
                                         'FOR MODULE-5',
                                         'FOR MODULE-6 AND 7'])].copy()

print(f"Records before cleaning: {len(df)}")
print(f"Records after removing unclassified modules: {len(df_clean)}")
print(f"Records removed: {len(df) - len(df_clean)}")

# Standardize course label names
course_map = {
    'Data Structures and Algorithms': 'DSA',
    'Operating Systems': 'OS',
    'Complex Variables and Linear Algebra': 'CVLA',
    'Microprocessors and Microcontrollers': 'MPMC',
    'Computer Architecture and Organisation': 'CAO'
}
df_clean['course_short'] =
df_clean['course_label'].map(course_map).fillna(df_clean['course_label'])

print(f"\nCourse distribution after cleaning:")
print(df_clean['course_short'].value_counts())
```

```
=====
DATA PREPROCESSING
=====
Records before cleaning: 452
Records after removing unclassified modules: 452
Records removed: 0

Course distribution after cleaning:
```

```
course_short
DSA      98
MPMC     98
CAO      94
CVLA     87
OS       75
Name: count, dtype: int64
```

```
# Extract text features from filename
def extract_text_from_filename(filename):
    """Convert filename into meaningful text tokens"""
    # Remove file extension
    text = re.sub(r'\.(pdf|pptx?|docx?)$', '', filename,
flags=re.IGNORECASE)
    # Replace separators with spaces
    text = re.sub(r'[-_\.\(\)\[\]\']', ' ', text)
    # Remove numbers that are standalone
    text = re.sub(r'\b\d{4,}\b', '', text) # remove long numbers like
dates
    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    return text.lower()

df_clean['text_features'] =
df_clean['filename'].apply(extract_text_from_filename)

# Add doc_type as a feature boost
df_clean['text_features'] = df_clean['text_features'] + ' ' +
df_clean['doc_type'] + ' ' + df_clean['file_format']

print("Sample text features extracted from filenames:")
for _, row in df_clean.head(5).iterrows():
    print(f"  Original: {row['filename']}")
    print(f"  Features: {row['text_features']}")
    print()
```

```
Sample text features extracted from filenames:
  Original: Data_Structures_and_Algorithm_Analysis_in_C.pdf
  Features: data structures and algorithm analysis in c notes pdf

  Original: CLRS-3rd Edition.pdf
  Features: clrs 3rd edition notes pdf

  Original: EBook-T1-Weiss-Datastructure(DSA).pdf
  Features: ebook t1 weiss datastructure dsa notes pdf

  Original: Data Structures and Algorithms – Narasimha Karumanchi.pdf
  Features: data structures and algorithms narasimha karumanchi notes
```

pdf

Original: EBook-R2-Advanced Algorithms-ThomasH-Cormen(DSA).pdf
Features: ebook r2 advanced algorithms thomash cormen dsa notes pdf

```
stop_words = set(stopwords.words('english'))
# Keep domain-specific words
domain_stopwords = {'pdf', 'pptx', 'ppt', 'docx', 'th', 'ppt', 'st',
                    'nd', 'rd'}
stop_words = stop_words | domain_stopwords

def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-z0-9\s', ' ', text)
    tokens = text.split()
    tokens = [t for t in tokens if t not in stop_words and len(t) > 2]
    return ' '.join(tokens)

df_clean['clean_text'] = df_clean['text_features'].apply(clean_text)

print("Sample cleaned text:")
for _, row in df_clean.head(3).iterrows():
    print(f" {row['clean_text']}")
```

Sample cleaned text:

```
data structures algorithm analysis notes
clrs 3rd edition notes
ebook weiss datastructure dsa notes
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle('Document Distribution Analysis', fontsize=16,
fontweight='bold')

# Course distribution
course_counts = df_clean['course_short'].value_counts()
colors = ['#2196F3', '#4CAF50', '#FF9800', '#E91E63', '#9C27B0']
axes[0].bar(course_counts.index, course_counts.values, color=colors,
edgecolor='black', linewidth=0.5)
axes[0].set_title('Documents per Course', fontsize=13)
axes[0].set_xlabel('Course')
axes[0].set_ylabel('Number of Documents')
axes[0].tick_params(axis='x', rotation=15)
for i, v in enumerate(course_counts.values):
    axes[0].text(i, v + 0.5, str(v), ha='center', fontweight='bold')

# Doc type distribution
type_counts = df_clean['doc_type'].value_counts()
```



```

axes[1].pie(type_counts.values, labels=type_counts.index,
autopct='%1.1f%%',
            colors=['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4'],
            startangle=90, wedgeprops={'edgecolor': 'white',
'linewidth': 2})
axes[1].set_title('Document Types', fontsize=13)

plt.tight_layout()
plt.savefig('fig1_document_distribution.png', dpi=150,
bbox_inches='tight')
plt.show()
print("Figure 1 saved!")

```

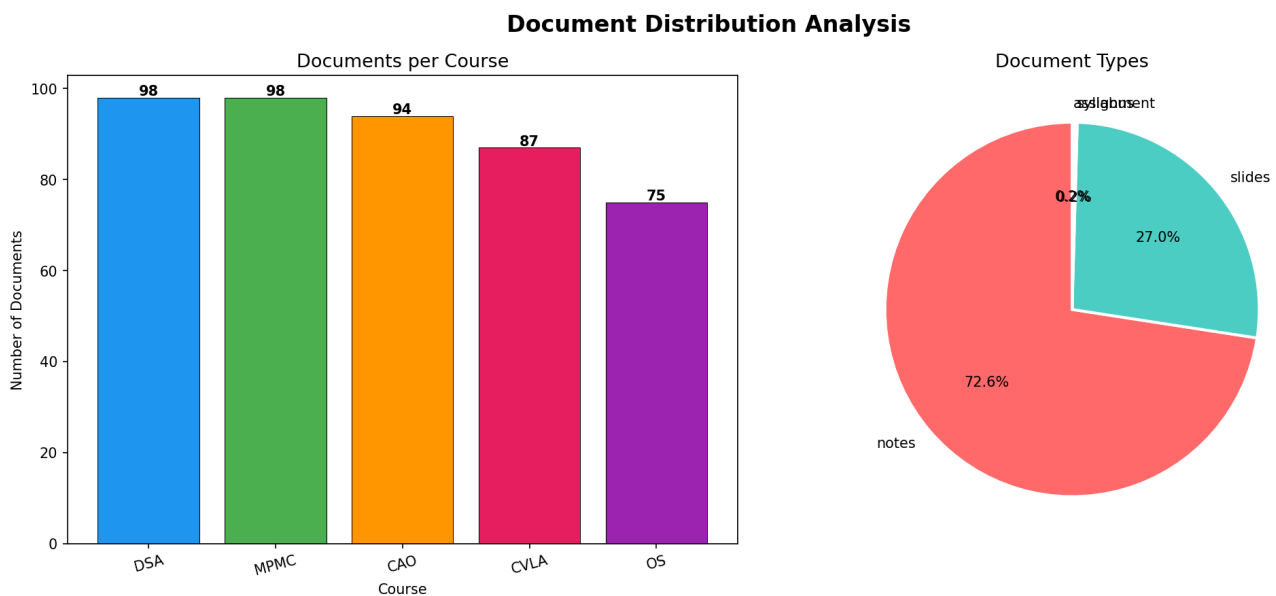


Figure 1 saved!

```

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Module Distribution per Course', fontsize=16,
fontweight='bold')
axes = axes.flatten()

courses = df_clean['course_short'].unique()
for idx, course in enumerate(sorted(courses)):
    course_df = df_clean[df_clean['course_short'] == course]
    mod_counts = course_df['module_label'].value_counts().sort_index()
    axes[idx].bar(range(len(mod_counts)), mod_counts.values,
                  color=colors[idx], edgecolor='black', linewidth=0.5)
    axes[idx].set_xticks(range(len(mod_counts)))
    axes[idx].set_xticklabels(mod_counts.index, rotation=45, ha='right',
    fontsize=9)
    axes[idx].set_title(f'{course} - Module Distribution', fontsize=11)

```

```

axes[idx].set_ylabel('Count')
for i, v in enumerate(mod_counts.values):
    axes[idx].text(i, v + 0.1, str(v), ha='center', fontsize=8)

# Hide unused subplot
if len(courses) < 6:
    axes[-1].set_visible(False)

plt.tight_layout()
plt.savefig('fig2_module_distribution.png', dpi=150,
bbox_inches='tight')
plt.show()
print("Figure 2 saved!")

```

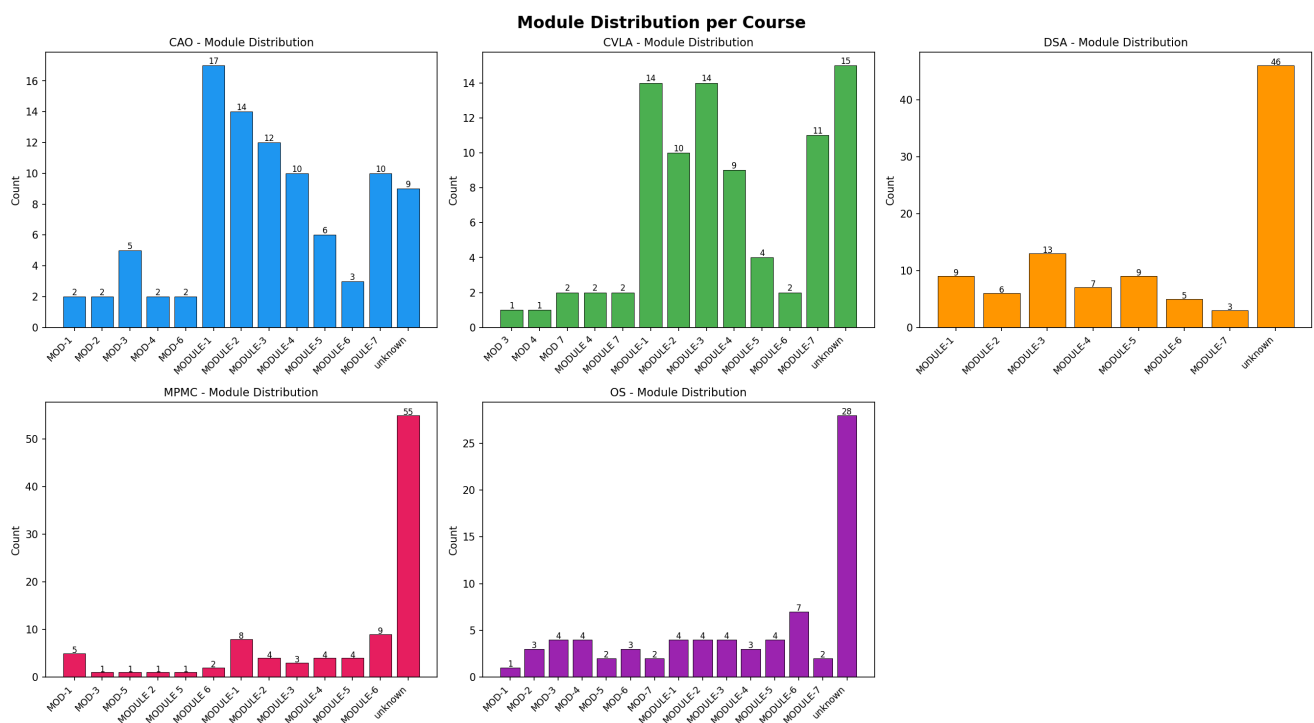


Figure 2 saved!

```

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Word Clouds per Course', fontsize=16, fontweight='bold')
axes = axes.flatten()

wc_colors = ['Blues', 'Greens', 'Oranges', 'RdPu', 'Purples']
for idx, course in enumerate(sorted(courses)):
    course_text = ' '.join(df_clean[df_clean['course_short'] == course]
['clean_text'])
    if course_text.strip():
        wc = WordCloud(width=400, height=250, background_color='white',
            colormap=wc_colors[idx], max_words=50,

```

```

        collocations=False).generate(course_text)
        axes[idx].imshow(wc, interpolation='bilinear')
        axes[idx].set_title(f'{course}', fontsize=12, fontweight='bold')
        axes[idx].axis('off')

if len(courses) < 6:
    axes[-1].set_visible(False)

plt.tight_layout()
plt.savefig('fig3_wordclouds.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 3 saved!")

```

Word Clouds per Course

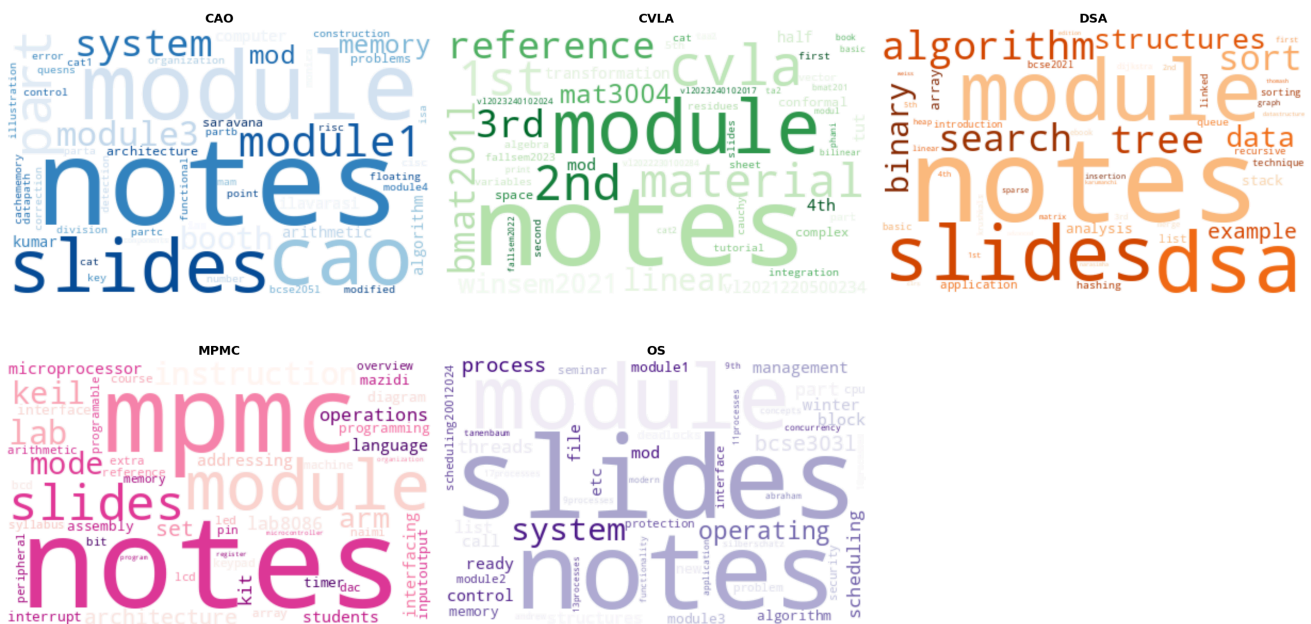


Figure 3 saved!

```

fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Top 10 TF-IDF Terms per Course', fontsize=16,
fontweight='bold')
axes = axes.flatten()

for idx, course in enumerate(sorted(courses)):
    course_docs = df_clean[df_clean['course_short'] == course]
    ['clean_text'].tolist()
    if len(course_docs) > 1:
        tfidf = TfidfVectorizer(max_features=500, ngram_range=(1,2))
        tfidf_matrix = tfidf.fit_transform(course_docs)
        mean_scores = np.array(tfidf_matrix.mean(axis=0)).flatten()
        top_indices = mean_scores.argsort()[-10:][::-1]
        top_terms = [tfidf.get_feature_names_out()[i] for i in

```

```

top_indices]
    top_scores = [mean_scores[i] for i in top_indices]

    axes[idx].barh(top_terms[::-1], top_scores[::-1],
color=colors[idx], edgecolor='black', linewidth=0.3)
    axes[idx].set_title(f'{course} - Top Terms', fontsize=11)
    axes[idx].set_xlabel('Mean TF-IDF Score')

if len(courses) < 6:
    axes[-1].set_visible(False)

plt.tight_layout()
plt.savefig('fig4_tfidf_terms.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 4 saved!")

```



Figure 4 saved!

```

print("=" * 55)
print("STATISTICAL SUMMARY")
print("=" * 55)

summary = df_clean.groupby('course_short').agg(
    Total_Documents=('filename', 'count'),

```

```

Unique_Modules=('module_label', 'nunique'),
PDF_Count=('file_format', lambda x: (x == 'pdf').sum()),
PPTX_Count=('file_format', lambda x: (x == 'pptx').sum()),
Notes_Count=('doc_type', lambda x: (x == 'notes').sum()),
Slides_Count=('doc_type', lambda x: (x == 'slides').sum()),
).reset_index()

print(summary.to_string(index=False))

# Correlation: doc_type vs file_format
print(f"\nDoc Type vs File Format Cross-tabulation:")
print(pd.crosstab(df_clean['doc_type'], df_clean['file_format']))

```

```

=====
STATISTICAL SUMMARY
=====
course_short  Total_Documents  Unique_Modules  PDF_Count  PPTX_Count
Notes_Count  Slides_Count
69          CAO              94              13          69          24
85          CVLA              87              13          85          2
69          DSA              98              8          66          29
70          MPMC              98              13          71          27
35          OS               75              15          35          40

```

Doc Type vs File Format Cross-tabulation:

```

file_format  docx  pdf  pptx
doc_type
assignment      1    0    0
notes           3  325    0
slides          0    0  122
syllabus        0    1    0

```

```

print("=" * 50)
print("MODEL PREPARATION")
print("=" * 50)

```

```

X = df_clean['clean_text']
y_course = df_clean['course_short']
y_module = df_clean['module_label']

```

```

# Filter modules with at least 3 samples for reliable training
module_counts = y_module.value_counts()
valid_modules = module_counts[module_counts >= 3].index

```

```

df_model = df_clean[df_clean['module_label'].isin(valid_modules)].copy()

X_mod = df_model['clean_text']
y_mod = df_model['module_label']
y_crs = df_model['course_short']

print(f"Samples for course classification: {len(y_course)}")
print(f"Samples for module classification: {len(y_mod)}")
print(f"\nCourse class distribution:\n{y_course.value_counts()}")
print(f"\nModule class distribution (top
10):\n{y_mod.value_counts().head(10)}")

# Train-test split
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X, y_course, test_size=0.2, random_state=42, stratify=y_course)

X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_mod, y_mod, test_size=0.2, random_state=42, stratify=y_mod)

print(f"\nTrain/Test split (80/20):")
print(f"  Course - Train: {len(X_train_c)}, Test: {len(X_test_c)}")
print(f"  Module - Train: {len(X_train_m)}, Test: {len(X_test_m)}")

```

```

=====
MODEL PREPARATION
=====

Samples for course classification: 452
Samples for module classification: 438

Course class distribution:
course_short
DSA      98
MPMC     98
CAO      94
CVLA     87
OS       75
Name: count, dtype: int64

Module class distribution (top 10):
module_label
unknown      153
MODULE-1     52
MODULE-3     46
MODULE-2     38
MODULE-4     33
MODULE-5     27
MODULE-6     26
MODULE-7     26
MOD-3        10

```

```
MOD-1      8
Name: count, dtype: int64
```

```
Train/Test split (80/20):
  Course - Train: 361, Test: 91
  Module - Train: 350, Test: 88
```

```
print("=" * 55)
print("TASK 1: COURSE IDENTIFICATION")
print("=" * 55)

models = {
    'Naive Bayes': Pipeline([
        ('tfidf', TfidfVectorizer(ngram_range=(1,2), max_features=3000,
sublinear_tf=True)),
        ('clf', MultinomialNB(alpha=0.1))
    ]),
    'Logistic Regression': Pipeline([
        ('tfidf', TfidfVectorizer(ngram_range=(1,2), max_features=3000,
sublinear_tf=True)),
        ('clf', LogisticRegression(max_iter=1000, C=1.0,
random_state=42))
    ]),
    'SVM (LinearSVC)': Pipeline([
        ('tfidf', TfidfVectorizer(ngram_range=(1,2), max_features=3000,
sublinear_tf=True)),
        ('clf', LinearSVC(C=1.0, random_state=42, max_iter=2000))
    ])
}

course_results = {}
for name, model in models.items():
    model.fit(X_train_c, y_train_c)
    y_pred = model.predict(X_test_c)
    acc = accuracy_score(y_test_c, y_pred)
    f1 = f1_score(y_test_c, y_pred, average='weighted')
    cv_scores = cross_val_score(model, X, y_course, cv=5,
scoring='f1_weighted')
    course_results[name] = {
        'model': model, 'accuracy': acc, 'f1': f1,
        'cv_mean': cv_scores.mean(), 'cv_std': cv_scores.std(),
        'y_pred': y_pred
    }
    print(f"\n{name}:")
    print(f"  Accuracy:          {acc:.4f} ({acc*100:.1f}%)")
    print(f"  F1 (weighted):     {f1:.4f}")
    print(f"  CV F1 (5-fold):   {cv_scores.mean():.4f} ±
{cv_scores.std():.4f}")
```

```
=====
TASK 1: COURSE IDENTIFICATION
=====
```

Naive Bayes:

```
Accuracy:      0.7802 (78.0%)
F1 (weighted): 0.7812
CV F1 (5-fold): 0.7209 ± 0.0628
```

Logistic Regression:

```
Accuracy:      0.7912 (79.1%)
F1 (weighted): 0.7898
CV F1 (5-fold): 0.7134 ± 0.0632
```

SVM (LinearSVC):

```
Accuracy:      0.7912 (79.1%)
F1 (weighted): 0.7919
CV F1 (5-fold): 0.7223 ± 0.0694
```

```
print("=" * 55)
print("TASK 2: MODULE IDENTIFICATION")
print("=" * 55)

module_results = {}
for name, _ in models.items():
    model = Pipeline([
        ('tfidf', TfidfVectorizer(ngram_range=(1,2), max_features=3000,
sublinear_tf=True)),
        ('clf', LinearSVC(C=1.0, random_state=42, max_iter=2000)
            if 'SVM' in name else
            LogisticRegression(max_iter=1000, random_state=42)
            if 'Logistic' in name else
            MultinomialNB(alpha=0.1))
    ])
    model.fit(X_train_m, y_train_m)
    y_pred = model.predict(X_test_m)
    acc = accuracy_score(y_test_m, y_pred)
    f1 = f1_score(y_test_m, y_pred, average='weighted')
    module_results[name] = {
        'model': model, 'accuracy': acc, 'f1': f1, 'y_pred': y_pred
    }
    print(f"\n{name}:")
    print(f"  Accuracy:      {acc:.4f} ({acc*100:.1f}%)")
    print(f"  F1 (weighted): {f1:.4f}")
```

```
=====
TASK 2: MODULE IDENTIFICATION
```


=====

Naive Bayes:

Accuracy: 0.4886 (48.9%)

F1 (weighted): 0.4712

Logistic Regression:

Accuracy: 0.3636 (36.4%)

F1 (weighted): 0.2872

SVM (LinearSVC):

Accuracy: 0.4545 (45.5%)

F1 (weighted): 0.4715

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle('Model Performance Comparison', fontsize=15,
fontweight='bold')

model_names = list(course_results.keys())
short_names = ['Naive\nBayes', 'Logistic\nRegression',
'SVM\n(LinearSVC)']
bar_colors = ['#2196F3', '#4CAF50', '#FF9800']
x = np.arange(len(model_names))
width = 0.35

# Course classification
c_acc = [course_results[m]['accuracy'] for m in model_names]
c_f1 = [course_results[m]['f1'] for m in model_names]
axes[0].bar(x - width/2, c_acc, width, label='Accuracy',
color=bar_colors, alpha=0.8, edgecolor='black')
axes[0].bar(x + width/2, c_f1, width, label='F1-Score',
color=bar_colors, alpha=0.5, edgecolor='black', hatch='//')
axes[0].set_xticks(x); axes[0].set_xticklabels(short_names)
axes[0].set_ylim(0, 1.15); axes[0].set_ylabel('Score')
axes[0].set_title('Task 1: Course Identification')
axes[0].legend(); axes[0].grid(axis='y', alpha=0.3)
for i, (a, f) in enumerate(zip(c_acc, c_f1)):
    axes[0].text(i - width/2, a + 0.02, f'{a:.2f}', ha='center',
    fontsize=9, fontweight='bold')
    axes[0].text(i + width/2, f + 0.02, f'{f:.2f}', ha='center',
    fontsize=9, fontweight='bold')

# Module classification
m_acc = [module_results[m]['accuracy'] for m in model_names]
m_f1 = [module_results[m]['f1'] for m in model_names]
axes[1].bar(x - width/2, m_acc, width, label='Accuracy',
color=bar_colors, alpha=0.8, edgecolor='black')
axes[1].bar(x + width/2, m_f1, width, label='F1-Score',
color=bar_colors, alpha=0.5, edgecolor='black', hatch='//')
```

```

axes[1].set_xticks(x); axes[1].set_xticklabels(short_names)
axes[1].set_ylim(0, 1.15); axes[1].set_ylabel('Score')
axes[1].set_title('Task 2: Module Identification')
axes[1].legend(); axes[1].grid(axis='y', alpha=0.3)
for i, (a, f) in enumerate(zip(m_acc, m_f1)):
    axes[1].text(i - width/2, a + 0.02, f'{a:.2f}', ha='center',
    fontsize=9, fontweight='bold')
    axes[1].text(i + width/2, f + 0.02, f'{f:.2f}', ha='center',
    fontsize=9, fontweight='bold')

plt.tight_layout()
plt.savefig('fig5_model_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 5 saved!")

```

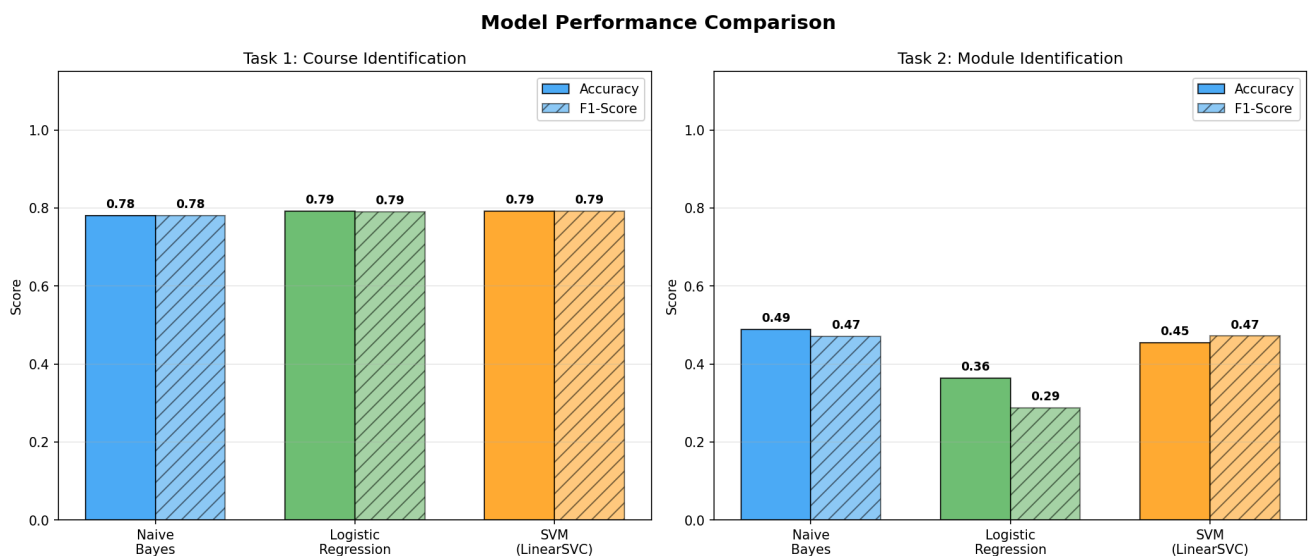


Figure 5 saved!

```

best_course_model_name = max(course_results, key=lambda x:
course_results[x]['f1'])
best_course_model = course_results[best_course_model_name]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle(f'Confusion Matrices – Best Model:
{best_course_model_name}',
    fontsize=14, fontweight='bold')

# Course confusion matrix
cm_c = confusion_matrix(y_test_c, best_course_model['y_pred'])
labels_c = sorted(y_course.unique())
sns.heatmap(cm_c, annot=True, fmt='d', cmap='Blues',
    xticklabels=labels_c, yticklabels=labels_c, ax=axes[0],

```

```

        linewidths=0.5, linecolor='gray')
axes[0].set_title('Course Identification', fontsize=12)
axes[0].set_xlabel('Predicted'); axes[0].set_ylabel('Actual')
axes[0].tick_params(axis='x', rotation=30)

# Module confusion matrix (top modules only for readability)
best_mod_model_name = max(module_results, key=lambda x:
module_results[x]['f1'])
best_mod_model = module_results[best_mod_model_name]
cm_m = confusion_matrix(y_test_m, best_mod_model['y_pred'])
labels_m = sorted(y_mod.unique())
sns.heatmap(cm_m, annot=True, fmt='d', cmap='Greens',
            xticklabels=labels_m, yticklabels=labels_m, ax=axes[1],
            linewidths=0.5, linecolor='gray')
axes[1].set_title('Module Identification', fontsize=12)
axes[1].set_xlabel('Predicted'); axes[1].set_ylabel('Actual')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.savefig('fig6_confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 6 saved!")

```

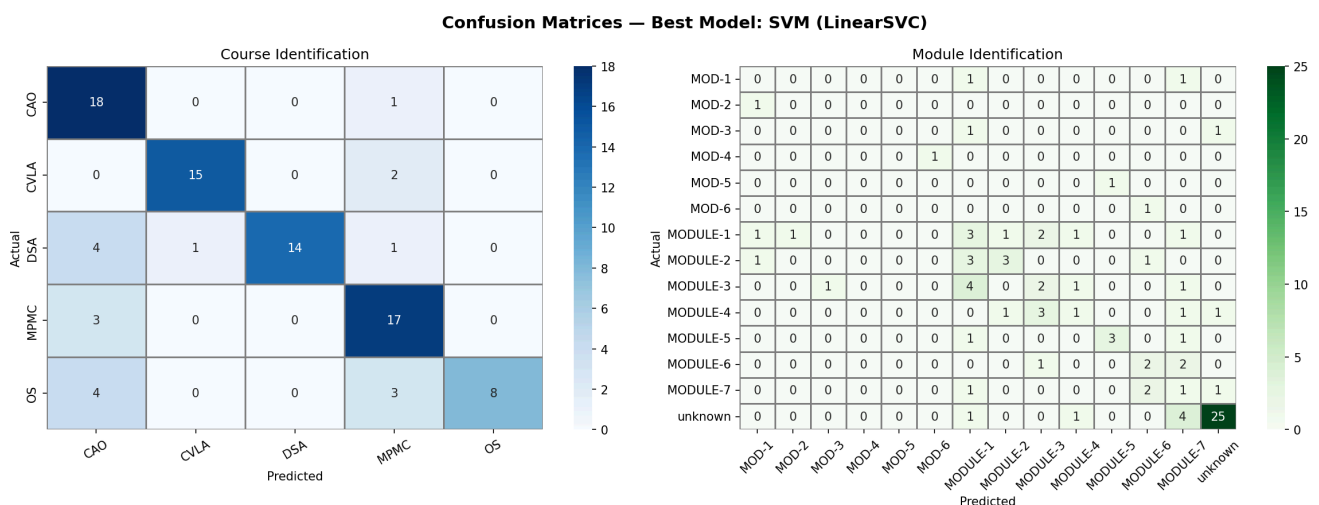


Figure 6 saved!

```

print("=" * 55)
print(f"BEST MODEL: {best_course_model_name}")
print("=" * 55)
print("\nCOURSE IDENTIFICATION - Classification Report:")
print(classification_report(y_test_c, best_course_model['y_pred']))

```

```
print("\nMODULE IDENTIFICATION - Classification Report:")
print(classification_report(y_test_m, best_mod_model['y_pred']))
```

```
=====
BEST MODEL: SVM (LinearSVC)
=====
```

```
COURSE IDENTIFICATION - Classification Report:
              precision    recall  f1-score   support

   CAO         0.62         0.95         0.75         19
   CVLA         0.94         0.88         0.91         17
   DSA          1.00         0.70         0.82         20
   MPMC         0.71         0.85         0.77         20
   OS           1.00         0.53         0.70         15

 accuracy              0.79         91
 macro avg           0.85         0.78         0.79         91
 weighted avg        0.85         0.79         0.79         91
```

MODULE IDENTIFICATION - Classification Report:
precision recall f1-score support

```

MOD-1         0.00         0.00         0.00          2
MOD-2         0.00         0.00         0.00          1
MOD-3         0.00         0.00         0.00          2
MOD-4         0.00         0.00         0.00          1
MOD-5         0.00         0.00         0.00          1
MOD-6         0.00         0.00         0.00          1
MODULE-1       0.20         0.30         0.24         10
MODULE-2       0.60         0.38         0.46          8
MODULE-3       0.25         0.22         0.24          9
MODULE-4       0.25         0.14         0.18          7
MODULE-5       0.75         0.60         0.67          5
MODULE-6       0.33         0.40         0.36          5
MODULE-7       0.08         0.20         0.12          5
unknown        0.89         0.81         0.85         31

 accuracy              0.45         88
 macro avg           0.24         0.22         0.22         88
 weighted avg        0.50         0.45         0.47         88
```

```
print("=" * 55)
print("LIVE PREDICTION DEMO")
print("=" * 55)
```

```

best_course_pipeline = best_course_model['model']
best_module_pipeline = best_mod_model['model']

def predict_document(filename):
    """Given a filename, predict its course and module"""
    text = extract_text_from_filename(filename)
    text = clean_text(text)

    course_pred = best_course_pipeline.predict([text])[0]
    module_pred = best_module_pipeline.predict([text])[0]

    print(f"\nDocument: {filename}")
    print(f"  → Predicted Course: {course_pred}")
    print(f"  → Predicted Module: {module_pred}")
    return course_pred, module_pred

# Test with sample documents
test_docs = [
    "Module-3-DSA-T7-Bubble Sort.pdf",
    "MEMORY MANAGEMENT PART 2 PAGING.pptx",
    "MPMC_4.2 8051 Timers and Counters.pdf",
    "Module 7 Matrices and System of equations.pdf",
    "Module 3-Part A_ISA CAO.pdf",
]

print("Testing prediction on sample documents:")
for doc in test_docs:
    predict_document(doc)

```

```

=====
LIVE PREDICTION DEMO
=====
Testing prediction on sample documents:

Document: Module-3-DSA-T7-Bubble Sort.pdf
  → Predicted Course: DSA
  → Predicted Module: MODULE-3

Document: MEMORY MANAGEMENT PART 2 PAGING.pptx
  → Predicted Course: OS
  → Predicted Module: MOD-5

Document: MPMC_4.2 8051 Timers and Counters.pdf
  → Predicted Course: MPMC
  → Predicted Module: unknown

Document: Module 7 Matrices and System of equations.pdf
  → Predicted Course: CVLA
  → Predicted Module: MODULE-7

```

Document: Module 3-Part A_ISA CAO.pdf

→ Predicted Course: CAO

→ Predicted Module: MOD-3

```
print("\n" + "=" * 55)
print("FINAL RESULTS SUMMARY")
print("=" * 55)

results_df = pd.DataFrame({
    'Model': model_names,
    'Course_Accuracy': [f"{course_results[m]['accuracy']*100:.1f}%" for
m in model_names],
    'Course_F1': [f"{course_results[m]['f1']:.4f}" for m in
model_names],
    'Module_Accuracy': [f"{module_results[m]['accuracy']*100:.1f}%" for
m in model_names],
    'Module_F1': [f"{module_results[m]['f1']:.4f}" for m in
model_names],
    'CV_Score': [f"{course_results[m]['cv_mean']:.4f} ±
{course_results[m]['cv_std']:.4f}" for m in model_names],
})
print(results_df.to_string(index=False))
print(f"\nBest model for Course ID: {best_course_model_name}")
print(f"Best model for Module ID: {best_mod_model_name}")
print("\nAll figures saved as PNG files in current directory.")
print("Project complete!")
```

```
=====
FINAL RESULTS SUMMARY
=====
      Model Course_Accuracy Course_F1 Module_Accuracy Module_F1
CV_Score
      Naive Bayes          78.0%    0.7812          48.9%    0.4712
0.7209 ± 0.0628
Logistic Regression      79.1%    0.7898          36.4%    0.2872
0.7134 ± 0.0632
      SVM (LinearSVC)      79.1%    0.7919          45.5%    0.4715
0.7223 ± 0.0694

Best model for Course ID: SVM (LinearSVC)
Best model for Module ID: SVM (LinearSVC)

All figures saved as PNG files in current directory.
Project complete!
```